



EAST WEST UNIVERSITY

PROJECT REPORT

Course Code: CSE325

Course Name: Operating System

Project Title:
IELTS/GRE Mock Exam Room Allocation System

Submitted by

Name	Student ID
Naheda Islam Ela	2024-3-60-080
Abdullah Al Junaid	2024-3-60-082

Submitted to

Shatabdi Roy Moon
Lecturer, Department of Computer Science and Engineering
East West University, Dhaka, Bangladesh

Section: 06
Semester: Summer 2026
Group: 06

Submission Date: 17-08-2026

Table of Contents

1. Introduction	3
2. Objectives	3
3. Problem Statement and Functional Requirements	3
4. Operating System Concepts Used	4
4.1 Process — fork()	4
4.2 Threads — pthread	4
4.3 Semaphores	4
4.4 Mutex and Condition Variable	4
4.5 Signal — SIGUSR1	4
5. System Design and Architecture	5
5.1 Data Model	5
5.2 Global Synchronization State	5
5.3 Program Flow	5
6. Source Code	8
7. Program Output	12
8. Dry Run — Full-Scale Walkthrough	13
9. Conclusion	14
10. References	14

1. Introduction

Universities and testing centres routinely run large, timed examinations such as IELTS or GRE mock tests, in which hundreds of students must be checked into fixed-capacity rooms, kept inside for the full duration of the exam, and accounted for at the end. This project, developed for CSE325 (Operating Systems), models exactly that scenario as a concurrent, multi-process, multi-threaded C++ program running on Linux. Rather than simulating the exam hall with ordinary function calls, the program deliberately uses real operating-system primitives — processes, threads, semaphores, a mutex, a condition variable, and a POSIX signal — so that each requirement of the exam-day workflow is enforced structurally by the operating system rather than by convention or careful ordering of code.

The program models 300 students distributed across 10 exam rooms of 30 seats each. Every student is represented by an independent thread; a separate child process, forked from the main program, plays the role of the invigilator who announces when the exam time is up. Because a real exam runs for several hours, the program compresses the exam duration to a small number of seconds (a configurable constant) purely so the simulation can be observed and graded quickly — the underlying synchronization logic is identical to what a full-scale, real-time version would use.

2. Objectives

The project was designed with the following learning and implementation objectives in mind:

- Demonstrate process creation and inter-process communication by using `fork()` to create an independent timer process that communicates back to the parent using a POSIX signal.
- Demonstrate multithreading by representing every student, and a dedicated monitor, as separate POSIX threads (pthread) running concurrently within a single process.
- Use counting semaphores to enforce two independent capacity constraints: how many students may occupy a room at once, and when students are permitted to enter at all.
- Use a mutex together with a condition variable to protect shared data and to guarantee that no student thread can leave its room before the exam has formally ended.
- Show, through a working program, how these primitives cooperate safely — without race conditions, lost updates, or deadlocks — to produce a correct final attendance count every time the program is run.

3. Problem Statement and Functional Requirements

The assignment specifies a small set of functional requirements that the program must satisfy, together with four operating-system concepts that must be used to satisfy them. The table below maps each requirement to the exact mechanism used to enforce it in the source code.

	Requirement	Enforced By
1	Every student must have a unique identifier.	<code>studentList[i].id = i</code> during setup — the array index doubles as the ID.
2	Every student must be assigned to exactly one exam room.	<code>studentList[i].room = i % NUM_ROOMS</code> , written once and never modified afterwards.
3	A room must never hold more students than its capacity.	One counting semaphore per room (<code>seatCounter[]</code>), initialised to <code>ROOM_CAPACITY</code> .
4	A student's room assignment must not change during the run.	room is read-only after setup; no thread ever writes to another thread's data.
5	Students may enter their room only after the exam officially starts.	A shared <code>goSignal</code> , initialised to 0, which every student thread waits on first.
6	No student may leave the room before the exam has ended.	<code>lock + examDone</code> hold every student thread until the invigilator signals the end.
7	The program must print a full attendance summary at the end.	<code>main()</code> prints <code>seatedInRoom[]</code> per room and the grand total after every thread has finished.

In addition, the assignment requires four specific operating-system concepts to be demonstrated together in a single program: a process created with `fork()`, one or more POSIX threads, at least one semaphore, and a mutex. All four are present and each plays a distinct, non-overlapping role, described in the next section.

4. Operating System Concepts Used

4.1 Process — `fork()`

`fork()` clones the running program into two independent processes that continue executing from the same point, distinguished only by the return value of `fork()` (zero in the child, the child's PID in the parent). In this project, the child process plays the role of the exam invigilator: it sleeps for the (simulated) exam duration and then sends a `SIGUSR1` signal back to the parent process using `kill()`, before terminating with `_exit(0)`. Using a genuinely separate process — rather than another thread — mirrors the real-world independence of an invigilator's clock from the students' activity, and gives the project a concrete demonstration of process creation and inter-process signalling, as required by the assignment.

4.2 Threads — `pthread`

Every one of the 300 students is represented by its own POSIX thread running `studentTask()`. A 301st thread, `watchClock()`, acts as the bridge between the operating-system signal and the rest of the program (explained in section 4.4). All threads share the same address space, which is exactly why the shared counters and flags they touch must be protected — this is the reason the mutex exists at all.

4.3 Semaphores

Two distinct counting semaphores are used, each enforcing a different rule:

- `seatCounter[NUM_ROOMS]` — one semaphore per room, initialised to `ROOM_CAPACITY` (30). A student calls `sem_wait()` on its room's semaphore before entering; if the count is already zero, the call blocks until a seat is freed by `sem_post()`. This is what makes exceeding a room's capacity structurally impossible, rather than merely unlikely.
- `goSignal` — a single shared semaphore initialised to 0. Every student thread calls `sem_wait()` on it as its very first action, so all 300 threads are guaranteed to block until `main()` explicitly posts to the semaphore once per student, which is how the program enforces that no one enters before the exam starts.

4.4 Mutex and Condition Variable

lock protects every piece of shared state that more than one thread might touch at the same time: the per-room attendance counters, the running total, and the `timeUp` flag. `examDone`, used together with the mutex, lets student threads sleep efficiently — rather than repeatedly polling a flag — until the exam has ended. Each student locks the mutex, checks `timeUp` inside a while loop, and calls `pthread_cond_wait()` if the exam is still running; `pthread_cond_wait()` atomically releases the mutex while the thread sleeps and reacquires it automatically upon waking. This atomicity is essential: if unlocking and sleeping were two separate steps, a wake-up could be sent in the gap between them and be lost forever. When the exam ends, `watchClock()` locks the mutex, sets `timeUp` to true, and calls `pthread_cond_broadcast()` so that every sleeping student wakes up at once.

4.5 Signal — SIGUSR1

`SIGUSR1` is one of only two signal numbers POSIX reserves for applications to define their own meaning (the other being `SIGUSR2`). Here it means exactly one thing: 'the exam timer has expired.' The handler registered for it, `alarmHandler()`, does the absolute minimum possible — it sets a single int flag, `alarmWentOff`, and returns immediately. This is deliberate: signal handlers may only safely call a very small, well-defined set of async-signal-safe functions, and neither `std::cout` nor any pthread locking function is on that list. Attempting to acquire the lock directly inside the handler could deadlock the program if the signal happened to arrive while the interrupted thread already held that same lock. Instead, a dedicated monitor thread polls the flag in an ordinary, signal-safe way and performs the real work — locking the mutex, setting `timeUp`, and broadcasting — once it is safe to do so.

5. System Design and Architecture

5.1 Data Model

The entire data model is a single plain struct, with one instance per student:

```
struct Student
{
```

```

    int id;
    int room;
    bool present;
};

```

5.2 Global Synchronization State

Variable	Type	Purpose
seatCounter[NUM_ROOMS]	sem_t	One counting semaphore per room; limits occupancy to ROOM_CAPACITY.
goSignal	sem_t	Gates all students until the exam officially starts.
lock	pthread_mutex_t	Guards seatedInRoom[], totalSeated, and timeUp.
examDone	pthread_cond_t	Lets student threads sleep until the exam ends, instead of busy-waiting.
timeUp	bool	Set to true only once, by the monitor thread, under lock.
alarmWentOff	volatile int	The only variable written directly inside the signal handler.

5.3 Program Flow

main() performs setup (assigning IDs and rooms, initialising every semaphore), registers the signal handler, then calls fork() to create the timer process. The parent process starts the monitor thread, creates all 300 student threads (each of which immediately blocks on goSignal), waits two seconds to simulate a pre-exam countdown, and then posts to goSignal once per student to open the gate. Every student thread then competes for a seat permit in its assigned room, records its own attendance under the mutex, and blocks on the condition variable until the invigilator process's timer expires and the monitor thread broadcasts the end of the exam. Finally, main() joins every thread, reaps the child process with wait(), and prints the attendance summary. The flowchart below traces this sequence end to end.

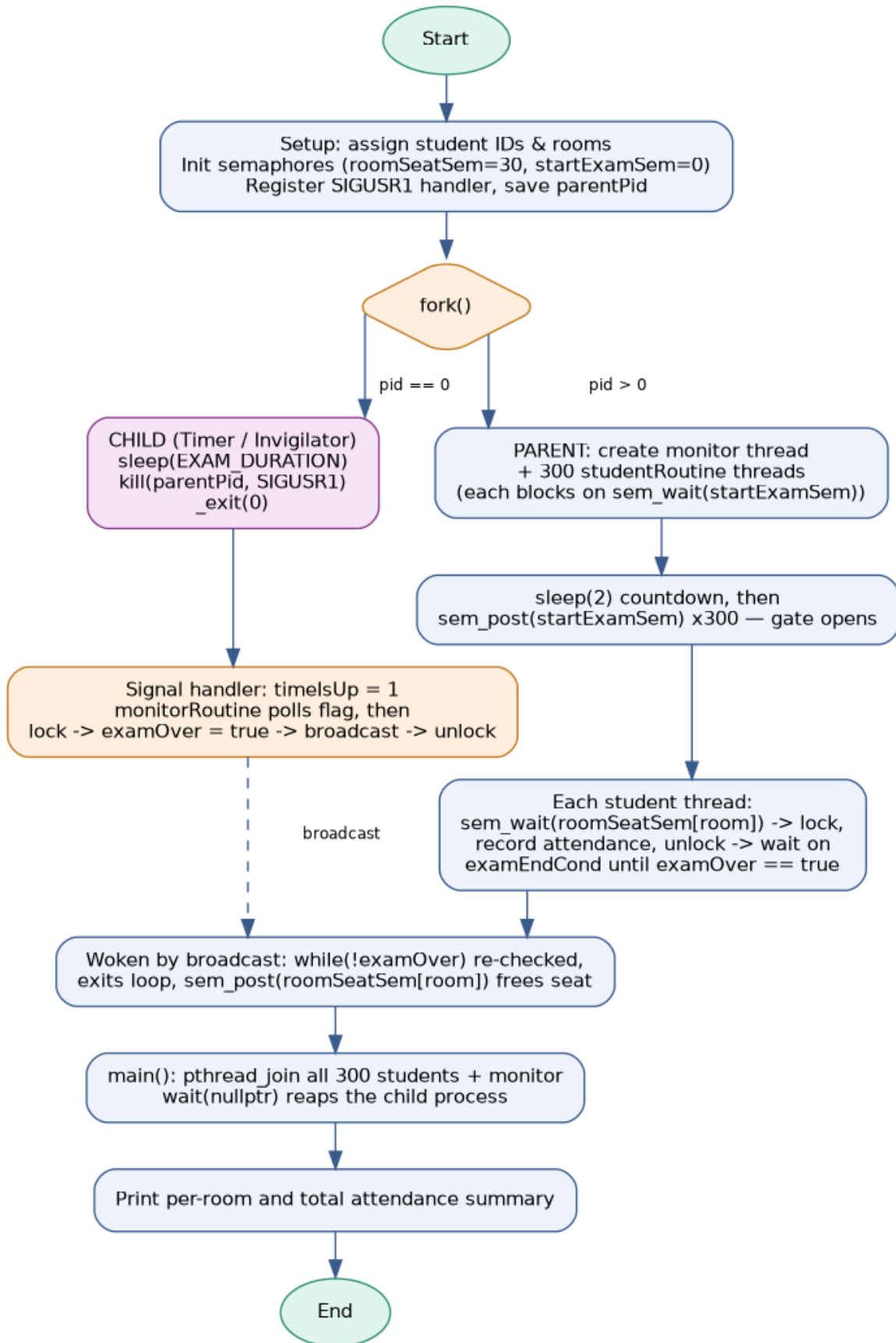


Figure 5.1 — Control-flow diagram of the exam hall simulation.

6. Source Code

The complete source code (exam_simple.cpp) is reproduced below. It compiles cleanly on Linux with:

```
#include <iostream>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <signal.h>
#include <cstdlib>

using namespace std;

const int TOTAL_STUDENTS = 300;
const int ROOM_CAPACITY = 30;
const int NUM_ROOMS = TOTAL_STUDENTS / ROOM_CAPACITY;
const int EXAM_TIME = 6;

struct Student
{
    int id;
    int room;
    bool present;
};

Student studentList[TOTAL_STUDENTS];
pthread_t studentThread[TOTAL_STUDENTS];

sem_t seatCounter[NUM_ROOMS];

sem_t goSignal;

pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t examDone = PTHREAD_COND_INITIALIZER;
bool timeUp = false;

int seatedInRoom[NUM_ROOMS];
int totalSeated = 0;

volatile int alarmWentOff = 0;

void alarmHandler(int signum)
{
    (void)signum;
    alarmWentOff = 1;
}

void *watchClock(void *)
{
    while (alarmWentOff == 0)
    {
```



```

        usleep(100000);
    }

    pthread_mutex_lock(&lock);
    timeUp = true;
    cout << "\n[INVIGILATOR] Time's up! Students may now leave their rooms.\n" << endl;
    pthread_cond_broadcast(&examDone);
    pthread_mutex_unlock(&lock);

    return nullptr;
}

void *studentTask(void *arg)
{
    int myId = *(int *)arg;
    int myRoom = studentList[myId].room;

    sem_wait(&goSignal);

    sem_wait(&seatCounter[myRoom]);

    pthread_mutex_lock(&lock);
    seatedInRoom[myRoom] = seatedInRoom[myRoom] + 1;
    totalSeated = totalSeated + 1;
    studentList[myId].present = true;
    cout << "Student " << myId << " entered Room " << (myRoom + 1)
         << " (" << seatedInRoom[myRoom] << "/" << ROOM_CAPACITY << " seats filled)" << endl;
    pthread_mutex_unlock(&lock);

    pthread_mutex_lock(&lock);
    while (timeUp == false)
    {
        pthread_cond_wait(&examDone, &lock);
    }
    pthread_mutex_unlock(&lock);

    cout << "Student " << myId << " is leaving Room " << (myRoom + 1) << endl;
    sem_post(&seatCounter[myRoom]);

    return nullptr;
}

int main()
{
    cout << "=====" << endl;
    cout << " IELTS / GRE Mock Exam - Room Allocation System" << endl;
    cout << " Total Students: " << TOTAL_STUDENTS
         << " | Rooms: " << NUM_ROOMS
         << " | Capacity per room: " << ROOM_CAPACITY << endl;
    cout << "=====\\n" << endl;

    for (int i = 0; i < TOTAL_STUDENTS; i++)
    {
        studentList[i].id = i;
        studentList[i].room = i % NUM_ROOMS;
        studentList[i].present = false;
    }
}

```

```

}

for (int r = 0; r < NUM_ROOMS; r++)
{
    sem_init(&seatCounter[r], 0, ROOM_CAPACITY);
    seatedInRoom[r] = 0;
}
sem_init(&goSignal, 0, 0);

signal(SIGUSR1, alarmHandler);

int myPid = (int)getpid();

int childPid = fork();

if (childPid < 0)
{
    cerr << "Error: fork() failed!" << endl;
    return 1;
}

if (childPid == 0)
{
    cout << "[TIMER PROCESS] Exam timer started. Exam will run for "
         << EXAM_TIME << " seconds (simulating 3 hours).\n" << endl;
    sleep(EXAM_TIME);
    kill(myPid, SIGUSR1);
    _exit(0);
}

pthread_t clockThread;
pthread_create(&clockThread, nullptr, watchClock, nullptr);

static int studentIds[TOTAL_STUDENTS];
for (int i = 0; i < TOTAL_STUDENTS; i++)
{
    studentIds[i] = i;
    pthread_create(&studentThread[i], nullptr, studentTask, &studentIds[i]);
}

cout << "Students are waiting outside... exam starts in 2 seconds." << endl;
sleep(2);
cout << "\n*** EXAM STARTED! Students may now enter their rooms. ***\n" << endl;

for (int i = 0; i < TOTAL_STUDENTS; i++)
{
    sem_post(&goSignal);
}

for (int i = 0; i < TOTAL_STUDENTS; i++)
{
    pthread_join(studentThread[i], nullptr);
}
pthread_join(clockThread, nullptr);

wait(nullptr);

```

```

cout << "\n===== " << endl;
cout << " EXAM ATTENDANCE SUMMARY" << endl;
cout << "===== " << endl;
for (int r = 0; r < NUM_ROOMS; r++)
{
    cout << "Room " << (r + 1) << ": "
          << seatedInRoom[r] << " / " << ROOM_CAPACITY << " students present" << endl;
}
cout << "-----" << endl;
cout << "Total Present: " << totalSeated << " / " << TOTAL_STUDENTS << endl;
cout << "Total Absent : " << (TOTAL_STUDENTS - totalSeated) << endl;
cout << "===== " << endl;

for (int r = 0; r < NUM_ROOMS; r++)
{
    sem_destroy(&seatCounter[r]);
}
sem_destroy(&goSignal);

return 0;
}

```

7. Program Output

The program was compiled and executed on Linux exactly as shown above, with EXAM_TIME left at its default value of 6. The opening and closing portions of an actual run are reproduced below; the omitted middle section repeats the same two message patterns — 'Student X entered Room Y' and, later, 'Student X is leaving Room Y' — for the remaining students.

7.1 Startup and first entries

```
=====
IELTS / GRE Mock Exam - Room Allocation System
Total Students: 300 | Rooms: 10 | Capacity per room: 30
=====

[TIMER PROCESS] Exam timer started. Exam will run for 6 seconds (simulating 3 hours).

Students are waiting outside... exam starts in 2 seconds.

*** EXAM STARTED! Students may now enter their rooms. ***

Student 0 entered Room 1 (1/30 seats filled)
Student 1 entered Room 2 (1/30 seats filled)
Student 2 entered Room 3 (1/30 seats filled)
Student 3 entered Room 4 (1/30 seats filled)
Student 4 entered Room 5 (1/30 seats filled)
Student 5 entered Room 6 (1/30 seats filled)
Student 6 entered Room 7 (1/30 seats filled)
Student 7 entered Room 8 (1/30 seats filled)
Student 8 entered Room 9 (1/30 seats filled)
Student 9 entered Room 10 (1/30 seats filled)
Student 10 entered Room 1 (2/30 seats filled)
Student 11 entered Room 2 (2/30 seats filled)
Student 12 entered Room 3 (2/30 seats filled)
```

7.2 Final departures and summary

```
=====
EXAM ATTENDANCE SUMMARY
=====
Room 1: 30 / 30 students present
Room 2: 30 / 30 students present
Room 3: 30 / 30 students present
Room 4: 30 / 30 students present
Room 5: 30 / 30 students present
Room 6: 30 / 30 students present
Room 7: 30 / 30 students present
Room 8: 30 / 30 students present
Room 9: 30 / 30 students present
Room 10: 30 / 30 students present
-----
Total Present: 300 / 300
Total Absent : 0
=====
```

Every one of the 300 students is accounted for, every room fills to exactly its 30-seat capacity, and Total Present exactly equals Total Absent's complement ($300 / 300$) — confirming that the semaphore, mutex, and condition-variable logic produced a correct result with no lost updates and no over-filled rooms, despite 300 threads running concurrently.

8. Dry Run — Full-Scale Walkthrough

This section traces the shared synchronization state through an actual run of the program at its real scale — 300 students, 10 rooms, 30 seats per room — the same configuration used to produce the output in section 7. Room assignment is $\text{id} \% \text{NUM_ROOMS}$, so students 0, 10, 20, ... share Room 1, students 1, 11, 21, ... share Room 2, and so on, with exactly 30 students landing in each of the 10 rooms. Because the capacity divides students evenly, every room's semaphore follows the same pattern, which is why a single column is enough to track all 10 rooms at once. "Mutex" is the lock's state and "Over" is the timeUp flag.

Step	Event	seatCounter (per room, cap 30)	Mutex	Over
1	Setup: 300 students get IDs 0–299 and $\text{room} = \text{id} \% 10$; semaphores initialised (each of the 10 rooms starts at 30, $\text{goSignal} = 0$).	30	free	false
2	<code>fork()</code> creates the timer process; parent continues.	30	free	false
3	300 student threads created; every one blocks instantly on <code>sem_wait(goSignal)</code> .	30	free	false
4	<code>main()</code> posts <code>goSignal</code> 300 times — the gate opens for everyone at once.	30	free	false
5	Students take seat permits: <code>sem_wait(seatCounter[room])</code> . As the 30 students assigned to a room enter, that room's semaphore count counts down from 30 to 0.	$30 \rightarrow 0$	free	false
6	Each student takes the lock, updates <code>seatedInRoom[]</code> and <code>totalSeated</code> , releases it — one at a time, so no update is ever lost. Final <code>totalSeated</code> reaches 300.	0	free	false
7	Each student re-locks, sees <code>timeUp == false</code> , calls <code>pthread_cond_wait</code> (sleeps, mutex released while asleep).	0	free	false
8	The timer process finishes its <code>sleep(EXAM_TIME)</code> , sends <code>SIGUSR1</code> to the parent, then exits.	0	free	false
9	Signal handler runs: <code>alarmWentOff = 1</code> — nothing else happens here.	0	free	false

Step	Event	seatCounter (per room, cap 30)	Mutex	Over
10	Monitor thread sees alarmWentOff, takes the lock, sets timeUp = true, broadcasts to every sleeping student, releases it.	0	free	true
11	All 300 students wake, re-check the while loop, exit it, and call sem_post(seatCounter[room]) — each room's semaphore climbs back from 0 to 30 as its 30 students leave.	0 → 30	free	true
12	main() joins all 300 student threads and the monitor thread, then wait(nullptr) reaps the timer process.	30	free	true
13	Summary printed: all 10 rooms show 30 / 30, Total Present 300 / 300 — matching the output captured in section 7.	30	free	true

9. Conclusion

This project implements a working, correctly synchronized simulation of an exam hall's room-allocation logic entirely in terms of operating-system primitives. A forked process plays an independent invigilator and communicates back through a POSIX signal; 300 student threads and one monitor thread run concurrently; two kinds of counting semaphores enforce the entry gate and the per-room capacity limit; and a mutex paired with a condition variable guarantees that shared counters are never corrupted and that no student can leave before the exam formally ends. The sample run in section 7 confirms that all 300 students are correctly seated, none are lost or double-counted, and the program terminates cleanly with every thread and the child process properly reaped. Beyond satisfying the assignment's functional requirements, the project demonstrates, in a small and observable program, why each primitive exists and how they compose safely into a single correct system.

10. References

1. W. R. Stevens and S. A. Rago, Advanced Programming in the UNIX Environment, 3rd ed., Addison-Wesley, 2013 — process creation, fork(), and signal handling.
2. The Open Group, POSIX.1-2017 (IEEE Std 1003.1), "pthread_create," "pthread_mutex_lock," "pthread_cond_wait," "sem_wait," "sem_post" reference pages.
3. Linux man-pages project — man7.org, sections 2 and 3, for fork(2), signal(7), pthread_create(3), sem_overview(7), and pthread_cond_wait(3).
4. D. P. Bovet and M. Cesati, Understanding the Linux Kernel, 3rd ed., O'Reilly Media — background on process and signal handling in Linux.
5. CSE325 (Operating Systems) course lecture materials and assignment specification, East West University, 2026.